Homework 4 Conceptual

CSCI 1470/2470: Deep Learning

Due Monday, March 18th, 2024 at 6:00pm EST

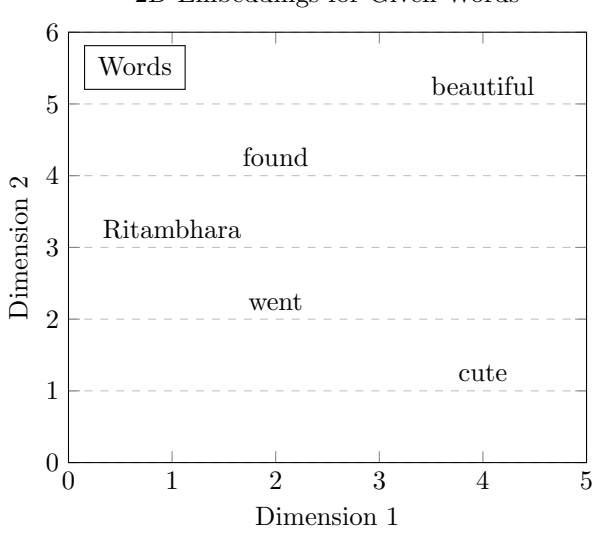
1 Conceptual Questions

1. Question 1: What are the dimensions of an embedding matrix? What do they represent?

   [ANSWER] The dimensions of an embedding matrix are determined by two key factors: the vocabulary size and the embedding size. Vocabulary size represents the total number of unique words (or tokens) in your dataset for which embeddings are being created. Each word in your vocabulary will have a unique vector associated with it. The vocabulary size thus corresponds to the number of rows in the embedding matrix.Embedding size is the size of the vector that you choose to represent each word. It's a parameter that you can set based on the complexity of your vocabulary and the specifics of your task. The embedding size determines the number of features you're using to represent the semantic properties of words. This corresponds to the number of columns in the embedding matrix.

Question 2: Given the following sentences (any relation to any real words is purely coincidental), plot reasonable embeddings in 2D for "Ritambhara", "went", "found", "cute", and "beautiful". (Hint: A simple graph with some clusters is fine.)

Ritambhara went scuba diving. Then Ritambhara found a beautiful seashell. A cute narwhal was next to the seashell.



Question 3 : What are some benefits of using RNNs over trigrams (or n-grams generally speaking?). Illustrate with an example.

[ANSWER] There are several benefits of using RNN over trigrams. Here are some aspects:
1.Handling of variable sequence lengths: RNNs can process input sequences of varying lengths, making them flexible for different text sizes or other sequential data. In contrast, n-grams are fixed in size and cannot easily adapt to longer or shorter contexts without altering the model structure.
2.Capturing long-term dependencies:RNNs are designed to retain information from earlier in the sequence, theoretically allowing them to capture long-range dependencies that might span beyond the immediate context of n-grams. For instance, an RNN can potentially understand the subject of a sentence even if it was introduced several words earlier, which might be missed by a trigram model.
3.Parameter efficiency: While the number of parameters in n-gram models grows exponentially with the size of the n-gram (which can lead to issues like data sparsity and model size blow-up), RNNs can be more parameter-efficient because they reuse the same weights for processing each element of the sequence.
4.Learning representations: RNNs can learn dense vector representations (embeddings) for words or other elements in the sequence, capturing nuanced semantic relationships. N-grams, on the other hand, rely on explicit counts or probabilities of fixed sequences, which might not capture deeper semantic similarities.
Consider the task of sentiment analysis on movie reviews for example. Using trigrams: You might capture phrases like "best movie ever" or "waste of time" to determine sentiment. However, this approach might miss nuanced expressions of sentiment that span more than three words or are not captured by the specific combinations seen in the training data. Using RNNs: The network can learn from the entire sequence of the review, understanding that phrases like "started slow but turned out great" or "not just good but fantastically entertaining" are positive, even though the sentiments are conveyed over several words with varying structures. This example highlights how RNNs' ability to consider longer contexts and generalize beyond exact word sequences seen during training offers a significant advantage over n-grams for complex language understanding tasks.

Question 4 :Why are LSTMs able to 'remember' information for longer timeframes than vanilla RNNs? Use the "dog" example in the class to illustrate this point.

[**ANSWER**] LSTM networks are a type of recurrent neural network architecture designed to address the vanishing gradient problem and the difficulty of learning long-range dependencies in traditional RNNs. LSTMs achieve this by incorporating a more complex memory cell structure with gating mechanisms.

Let's use the example "The dog that my family had when I was a child had a fluffy tail" to illustrate why LSTMs can remember information for longer timeframes compared to vanilla RNNs.

In a vanilla RNN: 1. Initially, the word "The" is fed into the RNN.

2. The RNN processes "The" and updates its internal hidden state.

3. The hidden state is then used to predict the next word, which might be "dog" or any other word based on the learned patterns.

4. If the RNN predicts "dog," the process continues with "dog" being fed into the network, and so on until the network generates "tail."

However, as the sequence length increases, the vanilla RNN encounters the vanishing gradient problem. The information about "dog" might get diluted or lost by the time the network predicts "tail." In this case, it might be particularly difficult for the vanilla RNN to remember the connection between "dog" and "tail" due to the long distance between them in the sequence.

Now, let's consider how an LSTM handles this example:

1. Initially, the word "The" is fed into the LSTM.

2. The LSTM processes "The" and updates its internal hidden state and cell state.

3. The hidden state and cell state are then used to predict the next word, which might be "dog" or any other word based on the learned patterns.

4. If the LSTM predicts "dog," the process continues with "dog" being fed into the network, and so on until the network generates "tail."

In contrast to vanilla RNNs, LSTMs have a more sophisticated memory cell structure with gating mechanisms. These gates allow LSTMs to selectively remember or forget information over long sequences.

In this example: - The cell state in the LSTM can retain information about "dog" even after processing several intermediate words. - The forget gate allows the LSTM to decide what information to discard from the cell state, preventing the vanishing gradient problem. - The input gate allows the LSTM to selectively update the cell state with new information, helping it remember relevant information for longer periods.

Therefore, even though there is a long distance between "dog" and "tail" in the sequence, LSTMs are better equipped to capture this long-range dependency compared to vanilla RNNs.

2 2470-only Questions

1. Question 1: The Gated Recurrent Unit (GRU) is another recurrent network cell that can, like the LSTM, retain information over long sequences. How is it able to do this? Draw the architectures (roughly) of the LSTM and GRU cells, and compare them. Also write the benefits and drawbacks of each (4-6 sentences).

   [ANSWER] GRU simplifies the LSTM architecture by combining the cell state and hidden state into a single state and using two gates (update and reset gates). The update gate decides how much of the past information to pass along to the future, and the reset gate determines how much of the past information to forget. This simplification allows GRUs to be more efficient than LSTMs in terms of computation.
   Benefits and Drawbacks: LSTMs have the advantage that they are very effective for a wide range of sequence prediction problems due to their ability to remember information for long periods and their capability to decide what information is relevant to retain or discard.On the other hand, they are computationally expensive and complex, which makes training longer and requires more computational resources. GRUs provide a more streamlined architecture that can reduce the complexity and improve the efficiency of training compared to LSTMs. They often perform comparably to LSTMs on many tasks but with less computational overhead. However, in some cases, especially in tasks that require modeling very long-term dependencies, GRUs might underperform compared to LSTMs due to their simplified structure.

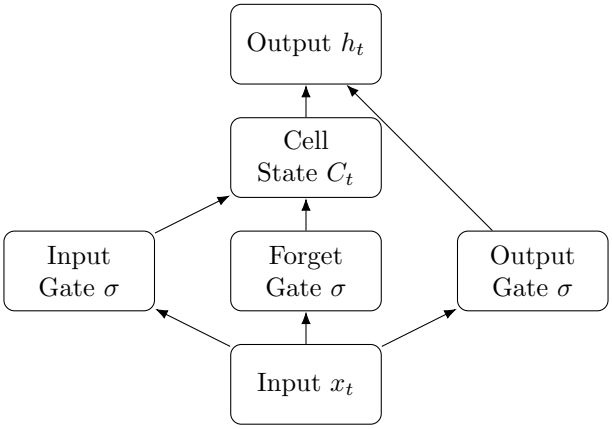


Figure 1: LSTM Cell Diagram

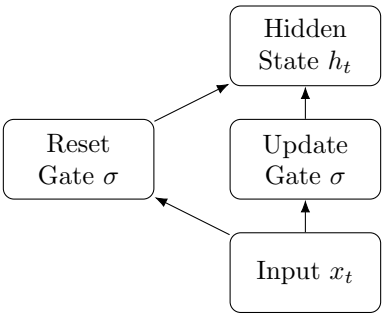


Figure 2: GRU Cell Diagram

(a) Question 2: While we have studied Convolutional Neural Networks (CNNs) in the context of 2D images, CNNs can also be used for 1D sequence modeling tasks, such as language modeling. Look up some papers that have attempted this, and that compare CNN language models to RNN language models (cite which papers you read). What appears to be the general consensus on the pros and cons of the two approaches?

[**ANSWER**] The paper "Comparative Study of CNN and RNN for Natural Language Processing" by Wenpeng Yin, Katharina Kann, Mo Yu, and Hinrich Schütze systematically compares CNNs and RNNs across various NLP tasks, aiming to provide guidance for DNN selection. It highlights that CNNs and RNNs offer complementary information for text classification, with the choice between them depending on the task's reliance on semantic understanding of the whole sequence or key phrases. The study also notes that CNNs might perform better in tasks where local, position-invariant features are crucial, whereas RNNs excel in tasks requiring modeling of long-range dependencies.